Open Source Architecture Appendix

Chen Xingqiang · github.com/chenxingqiang · module-level architecture with one project overview per page

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

CONTENTS · one architecture per page

- **TraceSmith** — GPU profiling & replay
- **YiRage** — RuntimeFusion
- **JAX-sklearn** — transparent JAX acceleration
- **Secret-Learn** — FL · SS · SL
- **YouTube-DNN** — candidate generation
- **Research CLI** — agent workflow
- **Questions-Gen** — three-stage training

READING GUIDE

Read top to bottom. Dashed frames are architecture zones; inner frames are subsystems; boxes are concrete modules.

Solid blue arrows are the primary path. Dashed purple arrows are fallback or feedback, not extra architecture layers.

TraceSmith — GPU Profiling & Replay Architecture

Application → _tracesmith.so → Capture / State / Format → Replay → GPU backends → trace outputs

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

PROJECT OVERVIEW

TraceSmith is a cross-platform GPU profiling and deterministic replay system for AI compiler, framework, and driver engineers.

Its C++17 core captures low-overhead execution events, stores compact SBT traces, and exports timelines for analysis and debugging.

APPLICATION · user entry

- **C++ Application** — in-process capture
- **Python Application** — in-process capture
- **C++ CLI** — tracesmith; profile · record · view
- **Python CLI** — tracesmith-cli; export · analyze · replay
- **External UI** — Perfetto UI; Chrome tracing

BINDINGS · pybind11 boundary

- **_tracesmith.so** — python/src/bindings.cpp; pybind11 native extension

CORE LIBRARY · C++17

Cluster Module

- **GPUTopology** — NVLink · PCIe · NVML
- **MultiGPUProfiler** — cross-device capture
- **TimeSync** — NTP / PTP / CUDA
- **NCCLTracker** — collective ops
- **ClockCorrelator** — clock alignment
- **CommAnalysis** — communication matrix

Capture Module

- **CUPTIProfiler** — NVIDIA CUDA
- **MCPTIProfiler** — MetaX MACA
- **MetalProfiler** — Apple Metal
- **MemoryProfiler** — GPU memory
- **AscendProfiler** — Huawei CANN
- **createProfiler()** — nsys · mcTracer; xctrace · msprof

State Module

- **GPUStateMachine** — state analysis
- **InstructionStream** — dependency graph
- **TimelineBuilder** — ordered events
- **TimelineViewer** — ASCII rendering
- **PerfettoExporter** — JSON
- **PerfettoProtoExporter** — Protobuf

Replay Module

- **ReplayEngine** — core orchestrator
- **StreamScheduler** — operation order
- **OperationExecutor** — launch replay
- **DeterminismChecker** — result compare
- **FrameCapture** — RenderDoc-style

Common Module

- **Types** — TraceEvent · EventType; KernelParams
- **RingBuffer** — lock-free SPSC
- **StackCapture** — libunwind
- **XRayImporter** — LLVM XRay
- **Timestamps** — ns precision
- **TraceMetadata** — DeviceInfo

Format Module

- **SBTWriter** — binary writer
- **SBTReader** — binary reader
- **.sbt** — header + events; counters

Configuration

- **ProfilerConfig** — buffer · overflow
- **ReplayConfig** — mode · speed
- **StackCaptureConfig** — stack policy
- **NCCLTrackerConfig** — collective policy

GDB Module

- **RSPHandler** — GDB protocol
- **ProcessController** — ptrace / Mach
- **GPUDebugEngine** — breakpoints · replay
- **tracesmith-gdbserver** — v0.10.0

replay

GPU BACKENDS · system interfaces

- **NVIDIA CUDA** — CUPTI SDK
- **Apple Metal** — Metal API
- **MetaX MACA** — MCPTI SDK
- **NCCL** — distributed
- **Nsight Systems** — nsys
- **mcTracer** — MetaX
- **xctrace** — Apple
- **libunwind** — host stacks
- **Perfetto SDK** — trace export

OUTPUTS · trace artifacts

- **trace.sbt** — binary format
- **trace.json** — Perfetto JSON
- **trace.perfetto** — Protobuf
- **graph.dot** — Graphviz
- **ASCII Timeline** — terminal
- **Comm Matrix** — heatmap JSON

Primary path: GPU event → RingBuffer → SBT/state → replay or Perfetto. Cluster, config, and GDB are adjacent capabilities.

YiRage (YPK) — RuntimeFusion Architecture

Engine owns schedule + KV; RF owns fusion identity; BackendRegistry searches and executes the same backend

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

PROJECT OVERVIEW

YiRage is a multi-backend LLM kernel compiler and runtime centered on graph superoptimization and RuntimeFusion.

It searches backend-specific fusion plans offline, then injects verified FusionCapsules into vLLM or SGLang decode steps.

INFERENCE ENGINE · scheduling owner

- **vLLM** — continuous batch · PagedAttention
- **SGLang** — RadixAttention · extend meta
- **Decode Metadata** — block_table · seq_lens · radix hit · SM budget · batch

RuntimeFusion · RF.step(inputs, meta)

- **RF.step** — select 0..N capsules per decode step
- **FusionCapsule** — MLP gate_up + down · yirage.core
- **FusionCapsule** — segment pipeline · gate_up → down
- **Native Fallback** — skip → engine native ops

SEARCH & PLAN · offline / warm-up

- **superoptimize** — KN / TB graph search
- **FusionPlan** — μGraph · profile + fingerprint
- **MuGraph cache** — shape bucket restore
- **Ray Coordinator** — optional distributed search
- **Serving Tiers** — seed_verify · full_tb · full_tb_ray

PYTHON API & BACKEND MANAGER

- **KNGraph / TBGraph** — transpiler → kernels
- **BackendRegistry** — search + execute · same backend
- **yirage.core** — Cython runtime
- **yirage.serving** — RF · Capsule · cert
- **MACA Config** — warp = 64 · mxcc
- **CPU MLIR JIT** — optional

BACKEND IMPLEMENTATIONS

- **CUDA** — nvcc · warp 32
- **MACA** — mxcc · warp 64
- **CPU** — OpenMP · MKL
- **MPS** — Apple GPU
- **Ascend** — CANN
- **ROCm** — AMD GPU
- **Triton** — DSL kernels
- **+ extra** — TPU · XPU · FPGA · MLIR

HARDWARE TARGETS

- **NVIDIA GPU**
- **MetaX C500/C550**
- **x86 / ARM CPU**
- **Apple Silicon**
- **Huawei Ascend**
- **AWS Neuron** — Intel XPU

fallback

Primary path: decode metadata → RF.step → FusionCapsule → BackendRegistry → selected backend and hardware.

JAX-sklearn (xlearn) — Transparent Acceleration Architecture

xlearn/__init__.py injects proxies; xlearn/_jax dispatches to JAX or the original sklearn estimator

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

PROJECT OVERVIEW

JAX-sklearn provides a drop-in scikit-learn API while transparently accelerating supported estimators with JAX.

Runtime proxies select JAX or native sklearn by algorithm, data size, and platform, preserving familiar inputs and outputs.

USER CODE · sklearn-compatible API

- **import xlearn as sklearn** — 100% call surface
- **Estimators** — linear_model · cluster; decomposition · preprocessing
- **Calls** — fit · predict · transform

COMPATIBILITY · xlearn/__init__.py

- **__getattr__** — lazy submodule import
- **_auto_jax_accelerate_module** — replace estimator classes
- **AcceleratorRegistry** — _accelerator.py
- **create_intelligent_proxy** — _proxy.py
- **monkey_patch_estimator** — install proxy class

JAX ACCELERATION · xlearn/_jax

- **EstimatorProxy** — fit-time dispatch
- **create_accelerated_estimator** — _accelerator.py
- **_universal_jax.py** — JIT · vmap
- **original_class** — sklearn fallback
- **_gp.py** — Gaussian process
- **_mlp.py** — neural net
- **_svm.py** — SVM kernels
- **_tree.py** — tree models

fallback

DATA MANAGEMENT · arrays and policy

- **_data_conversion.py** — NumPy ↔ JAX; dtype · shape
- **_config.py** — enable_jax; jax_platform
- **jax_auto_threshold** — complexity >= 1e8
- **detect_hardware** — install command; platform probe

HARDWARE · JAX extras

- **CPU** — jax-cpu · jaxlib
- **CUDA GPU** — jax-gpu; jax-cuda11
- **TPU** — jax-tpu
- **Apple Metal** — jax-metal; float32 · experimental

Primary path: sklearn call → injected proxy → JAX estimator. Small data, missing ops, or errors return through original_class.

Secret-Learn — Privacy-Preserving ML Architecture

secretlearn API → FL / SS / SL placement → JAX acceleration → SecretFlow devices

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

PROJECT OVERVIEW

Secret-Learn exposes sklearn-compatible machine learning across federated, secret-sharing, and split-learning modes.

It combines JAX acceleration with SecretFlow devices so each workflow can choose where plaintext, shares, and activations reside.

APPLICATION · multi-party use cases

- **Healthcare** — hospital collaboration, patient privacy
- **Finance** — cross-bank analysis, fraud detection
- **IoT / Edge** — edge intelligence, distributed learning
- **Research** — genomics, multi-institution
- **Government** — census analytics, cross-agency data

SKLEARN API · 191 algorithms

- **Linear Models** — 39 — Ridge · Lasso · SGD
- **Ensemble** — 18 — RF · XGBoost
- **Clustering** — 14 — KMeans · DBSCAN
- **Decomposition** — 14 — PCA · NMF · ICA
- **SVM** — 7 — SVC · SVR
- **Neural Networks** — 4 — MLP · BernoulliRBM
- **+ 95 More** — Neighbors · Trees, Naive Bayes

PRIVACY MODES · execution placement

FEDERATED LEARNING

- **Local PYU** — data stays local
- **HEU Aggregate** — secure averaging
- **FLLinearRegression** — federated_learning/
- **Horizontal FL** — 3–5×

SECRET SHARING

- **SPU MPC** — ciphertext compute
- **ABY3 / CHEETAH** — MPC protocols
- **SSPCA** — secret_sharing/
- **Maximum Privacy** — 1–2×

SPLIT LEARNING

- **Model Split** — across parties
- **HEU Activations** — protected tensors
- **SLMLPClassifier** — split_learning/
- **Vertical FL** — 2–4×

JAX ACCELERATION · xlearn bridge

- **JIT / XLA** — traced execution
- **jax.vmap** — batching · autodiff
- **xlearn Backend** — JAX-sklearn bridge
- **Smart Fallback** — original sklearn
- **CPU** — 1.5–2.5×
- **GPU** — 5–8×
- **TPU** — 9–15×
- **M1 / M2** — 2–4×

SECURE DEVICES · SecretFlow

SPU

- **MPC** — ABY3 · CHEETAH
- **Ciphertext** — zero plaintext leak

HEU

- **Schemes** — Paillier · CKKS
- **Aggregation** — gradient protection

TEE

- **Enclaves** — Intel SGX · AMD SEV
- **Occlum** — attestation

PYU

- **Party Local** — data ownership
- **JAX Local** — plaintext compute

INFRASTRUCTURE · runtime stack

- **SecretFlow** — Ray distributed, gRPC · DP-SGD
- **JAX / XLA** — CUDA / cuDNN, TPU runtime
- **sklearn** — NumPy · SciPy

Primary path: one sklearn call selects FL, SS, or SL, then runs on PYU, HEU, SPU, or TEE.

YouTube-DNN-RecSys — Candidate Generation Architecture

Python / Spark samples → TFRecords → dnn.py user tower → Spark or Python prediction

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

PROJECT OVERVIEW

YouTube-DNN-RecSys is a TensorFlow and Spark implementation of the candidate-generation stage of YouTube recommendations.

It turns behavior histories into TFRecords, learns user and item embeddings, and produces Top-K recall candidates for serving.

DATA PROCESSING · offline sample build

Python Pipeline

data2tfrecords.py
random training data

TFRecords
train / evaluation

read_tfrecords.py
dataset reader

Spark Pipeline

MakeDataOne.scala
distributed generation

MakeDataTwo.scala
TFRecords output

HDFS
dataset storage

STORAGE · datasets and checkpoints

data/tfrecords/
train/train.tfrecords
evaluation/evaluation.tfrecords

data/checkpoints/
ckpt/checkpoint
ckpt/eval · modelpath/

eval

MODEL TRAINING · dnn.py

Input
visit_items_index × 5
continuous_features × 16

Item Embedding
500 items × 16

Average Pooling
history vector

DNN Tower
Dense 64 + ReLU
Dropout · Dense 32 + ReLU
user vector 16

Softmax Output
500 classes
cross entropy · Top-K

PREDICTION · serving paths

PredictUserVector.scala
SavedModel
embedding_average → user_vector

ItemEmbeddingPredictor.scala
item embedding table
FeatureBuilder

load_dnn_model.py
SavedModel load
online / batch inference

UTILITIES · training support

tensor_board.py
TensorBoard

feature_column.py
feature spec

self_defined_network_layer.py
custom TF layers

Spark Base
BaseSparkLocal.scala
BaseSparkOnline.scala

Configuration
vars.prod.properties
vars.sit.properties

Primary path: behavior history → TFRecords → embedding average pool → user vector → item table → Top-K.

Research CLI — Agent Request Architecture

packages/cli → packages/core routing → tools / MCP / research modules → model providers

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

PROJECT OVERVIEW

Research CLI is a terminal-first AI agent workflow that connects interactive sessions to models, tools, and MCP services.

Its core router manages context and provider selection while research modules cover writing, bibliography, analysis, and submission.

CLI FRONTEND · packages/cli

- **research.tsx** — interactive TUI
- **commands/** — mcp.ts; skills.tsx · hooks.tsx
- **ui/commands/research** — arxivCommand; paper · investigate · submit
- **acpSession.ts** — ACP session; stdio transport

CORE ORCHESTRATION · packages/core

Session

- **researchChat.ts** — researchRequest.ts
- **research-config-manager.ts** — academic settings
- **config / memory** — context window

Routing

- **modelRouterService** — routing/modelRouterService.ts
- **defaultStrategy** — routingStrategy.ts
- **classifierStrategy** — gemma · numerical
- **compositeStrategy** — approval · fallback; override

TOOLS & RESEARCH MODULES

Workspace

- **edit.ts · grep.ts** — filesystem changes
- **shell / ls** — local execution

MCP

- **mcp-client.ts** — server sessions
- **mcp-client-manager.ts** — resource routing

analysis/

- **bibliography-manager** — citations
- **experiment-code-generator** — Python · R · Julia
- **research-data-analyzer** — statistics · ML

bibliography/

- **arxiv-mcp-client** — arXiv search
- **google-scholar-client** — scholar search
- **literature-investigator** — keyword-generator; reference-classifier

submission/

- **latex-manager** — project compile
- **journal-matcher** — venue selection
- **arxiv-latex-extractor** — template extract

loop

MODEL PROVIDERS

- **Gemini / Vertex**
- **OpenAI**
- **Anthropic**
- **Baidu ERNIE**
- **DeepSeek**
- **Qwen**
- **Groq / Mistral** — Cohere · Moonshot

EMBEDDED RUNTIMES

- **packages/sdk** — library embedding
- **packages/a2a-server** — agent executor; HTTP task API
- **vscode-ide-companion** — IDE companion
- **packages/devtools** — development tools

Primary path: prompt → research.tsx → modelRouterService → provider, with tools and research modules supplying observations.

Questions-Gen — Three-Stage Training Architecture

Question data → Qwen3-14B SFT → GRPO reward → DeepSeek-R1 distillation → validation → publish

Primary flow
Fallback / feedback
Dashed = architecture zone
Box = module + responsibility

PROJECT OVERVIEW

Questions-Gen is a Qwen3-14B system for generating and evaluating mathematical competition problems through staged training.

The pipeline combines supervised fine-tuning, GRPO reward optimization, DeepSeek-R1 distillation, validation, and local deployment.

TRAINING DATA · documented mixture

- **Historical** — 50% — QuestionDataPreparer.py
- **Conditional Variants** — 30% — data_preparer.py
- **Innovative Types** — 20% — real_math_data_processor.py

Stage 1 · SFT

- **Qwen3-14B** — base checkpoint
- **QuestionsGenTrainer.py** — src/ + questions_gen/core/trainer.py
- **stage1-fp-merged** — Hugging Face

Stage 2 · GRPO

RewardCalculator.py

- **Difficulty** — 0.4 difficulty
- **Novelty** — 0.3
- **Rigor** — 0.2
- **Diversity** — 0.1

Optimization

- **GRPO Groups** — 8 problems/group
- **NoveltyConstraint.py** — duplication guard
- **stage2-merged-16bit** — variation generation

quality

Stage 3 · Distill

- **DeepSeekTeacher.py** — DeepSeek-R1
- **Teacher Signals** — difficulty · rigor · innovation · education
- **final-merged-16bit** — Hugging Face

VALIDATION · questions_gen/validation

- **model_validator.py** — single checkpoint
- **batch_validator.py** — algebra · calculus · geometry · number theory
- **quality_evaluator.py** — clarity · difficulty · originality

DEPLOYMENT · interfaces

- **main_cli.py** — validate · batch · quality
- **hf_utils.py** — three published stages
- **ollama_manager.py** — local inference
- **examples/** — quick_demo.py, demo_model_validation.py

Primary path: mixed problem set → SFT → GRPO reward and novelty → DeepSeek-R1 distillation → validate → Hugging Face / Ollama.